Carrera: **Tec. Sup. en Informática Aplicada**
Materia: **Programación II**
Comisión: **2-602 TT**

Proyecto: **JTEducation**
Título: **Alumno en todas sus capas**
Versión: **2026.10.26**

ALUMNO EN TODAS SUS CAPAS

Entidad, repositorio, servicio, controlador y vista, con sus validaciones y excepciones

Tabla de contenido

Introducción
- El recorrido de una petición

Capa de Base de datos: la tabla alumno

Capa de Entidad (Entity): la clase Alumno
- Las anotaciones de la entidad
- La entidad se valida a sí misma

Capa de Repositorio (Repository): acceso a datos
- Métodos que usamos

El formulario: AlumnoForm (DTO)
- Anotaciones de Bean Validation

Capa de Servicio (Service): las reglas de negocio
- Crear un alumno
- Reglas de negocio
- Eliminar un alumno

Capa de Controlador (Controller): conectar todo
- Rutas de Alumno

Capa de Vista (View): Thymeleaf

Validaciones: cuatro barreras
- Ejemplo: un DNI repetido

Excepciones
- Excepciones propias
- Quién lanza cada excepción y quién la atrapa
- Manejo centralizado: GlobalExceptionHandler
- Errores de conversión

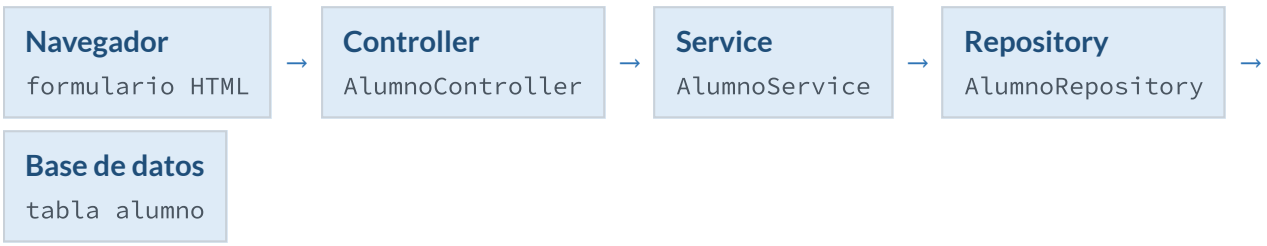
Resumen: crear un alumno de punta a punta

Introducción

En este apunte seguimos a una sola entidad, **Alumno**, desde la tabla de la base de datos hasta la página que ve el usuario. En cada capa mostramos el código real del proyecto y explicamos cada anotación. Al final juntamos dos temas que atraviesan todas las capas: las **validaciones** y las **excepciones**.

El recorrido de una petición

Cuando el usuario guarda el formulario de un alumno, la petición atraviesa las capas en este orden y la respuesta vuelve por el mismo camino:



Entre todas viaja el objeto **Alumno** (la entidad), y el formulario viaja como **AlumnoForm** (un DTO). La vista Thymeleaf es la que arma el HTML de la respuesta.

Capa	Clase de Alumno	Responsabilidad
Base de datos	Tabla **alumno** en *Escuela_DDL.sql*	Guardar los datos y garantizar claves y restricciones.
entity	**Alumno.java**	Representar una fila como objeto Java y validar sus propios datos.
repository	**AlumnoRepository.java**	Leer y escribir en la base sin escribir SQL.
dto	**AlumnoForm.java**	Recibir los datos del formulario y validarlos.
service	**AlumnoService.java**	Aplicar las reglas de negocio y manejar las transacciones.
controller	**AlumnoController.java**	Recibir la petición, llamar al servicio y elegir la vista.
templates	**alumnos/*.html**	Mostrar los datos al usuario (Thymeleaf).

Capa de Base de datos: la tabla alumno

Todo empieza en MySQL. La tabla `alumno` guarda los datos propios del alumno y tres claves foráneas que lo conectan con su usuario, su curso y su domicilio.

bbd/Escuela_DDL.sql (simplificado)

```
CREATE TABLE `alumno` (
  `id_alumno` INT NOT NULL AUTO_INCREMENT,
  `dni` VARCHAR(8) NOT NULL,                    -- 7 u 8 dígitos, como texto
  `nombre` VARCHAR(45) NOT NULL,
  `apellido` VARCHAR(45) NOT NULL,
  `fecha_nacimiento` DATE NOT NULL,
  `fecha_inscripcion` DATE NOT NULL,
  `usuario_id_usuario` INT NOT NULL,
  `curso_id_curso` INT NOT NULL,
  `domicilio_id_domicilio` INT NOT NULL,
  PRIMARY KEY (`id_alumno`),
  UNIQUE INDEX `dni_UNIQUE` (`dni` ASC),
  UNIQUE INDEX `usuario_id_usuario_UNIQUE` (`usuario_id_usuario` ASC),
  CONSTRAINT `fk_alumno_usuario1` FOREIGN KEY (`usuario_id_usuario`) REFERENCES `usuario`
(`id_usuario`),
  CONSTRAINT `fk_alumno_curso1` FOREIGN KEY (`curso_id_curso`) REFERENCES `curso` (`id_curso`),
  CONSTRAINT `fk_alumno_domicilio1` FOREIGN KEY (`domicilio_id_domicilio`) REFERENCES `domicilio`
(`id_domicilio`)
) ENGINE = InnoDB;
```

Las líneas resaltadas son la **última barrera** de validación: aunque el código Java fallara, la base nunca acepta dos alumnos con el mismo DNI ni dos alumnos con la misma cuenta de usuario. Ese `UNIQUE` sobre `usuario_id_usuario` es lo que convierte la relación con el usuario en **1:1**.

Capa de Entidad (Entity): la clase Alumno

La entidad es la clase Java que representa una fila de la tabla. **JPA** (con Hibernate como implementación) se encarga de convertir objetos en filas y filas en objetos. Esto se llama **mapeo objeto-relacional (ORM)**.

entity/Alumno.java (atributos y relaciones)

```java
@Entity
@Getter
@NoArgsConstructor
public class Alumno {

    private static final String DNI_REGEX = "\\d{7,8}"; // DNI argentino: 7 u 8 dígitos

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idAlumno; // camelCase en Java; Spring lo traduce solo a la columna id_alumno

    private String dni;
    private String nombre;
    private String apellido;
    private LocalDate fechaNacimiento;    // -> fecha_nacimiento
    private LocalDate fechaInscripcion;   // -> fecha_inscripcion

    // RELACIÓN 1:1. cascade = ALL: al guardar/borrar el alumno, se guarda/borra su domicilio.
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "domicilio_id_domicilio", nullable = false)
    private Domicilio domicilio;

    // RELACIÓN 1:1. Sin cascade: el usuario se crea primero (con su rol y su contraseña
hasheada).
    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "usuario_id_usuario", nullable = false)
    private Usuario usuario;

    // RELACIÓN 1:N. Muchos alumnos pertenecen a un curso.
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "curso_id_curso", nullable = false)
    private Curso curso;
```

Las anotaciones de la entidad

Anotación	De dónde viene	Qué hace
`@Entity`	JPA	Marca la clase como entidad: cada objeto es una fila de la tabla **alumno**.
`@Id`	JPA	Indica cuál es la clave primaria.
`@GeneratedValue(IDENTITY)`	JPA	El id lo genera la base (**AUTO_INCREMENT**); por eso no hay setter de id.
`@OneToOne`	JPA	Relación uno a uno: un alumno tiene un domicilio y un usuario.

Anotación	De dónde viene	Qué hace
`@ManyToOne`	JPA	Relación muchos a uno: muchos alumnos pertenecen a un curso. Del lado del alumno queda la clave foránea.
`@JoinColumn`	JPA	Nombre de la columna de la clave foránea. **nullable = false**: es obligatoria.
`fetch = LAZY`	JPA	La relación se trae de la base recién cuando se usa (por ejemplo, al llamar a **getCurso().getAnio()**).
`cascade = ALL`	JPA	Lo que se hace con el alumno (guardar, borrar) se repite en su domicilio.
`optional = false`	JPA	La relación no puede quedar vacía.
`@Getter`	Lombok	Genera todos los **get** al compilar.
`@NoArgsConstructor`	Lombok	Genera el constructor vacío que JPA necesita para crear objetos al leer de la base.

No usamos `@Setter` de Lombok a propósito: cada setter lo escribimos a mano para que **valide** antes de asignar.

La entidad se valida a sí misma

Una regla de la programación orientada a objetos es que un objeto nunca debería quedar en un estado inválido. Por eso el constructor y los setters pasan cada dato por un método `validarX` privado. Si el dato es inválido, se lanza una `IllegalArgumentException` y el objeto no se modifica.

entity/Alumno.java (constructor, setter y validaciones)

```
    public Alumno(String dni, String nombre, String apellido, LocalDate fechaNacimiento,
            LocalDate fechaInscripcion, Domicilio domicilio, Usuario usuario, Curso curso) {
        this.dni = validarDni(dni);
        this.nombre = validarNombre(nombre);
        // ... el resto de los atributos, igual
    }

    public void setDni(String dni) {
        this.dni = validarDni(dni);
    }

    // ===== Validaciones privadas: las comparten el constructor y los setters =====

    private static String validarDni(String dni) {
        String limpio = (dni == null) ? "" : dni.trim();
        if (!limpio.matches(DNI_REGEX)) {
            throw new IllegalArgumentException("El DNI debe tener 7 u 8 dígitos.");
        }
        return limpio;
    }

    private static LocalDate validarFechaNacimiento(LocalDate fecha) {
        if (fecha == null || !fecha.isBefore(LocalDate.now())) {
            throw new IllegalArgumentException("La fecha de nacimiento debe ser anterior a hoy.");
        }
        return fecha;
    }

    private static String validarNombre(String nombre) {
        return Validaciones.texto(nombre, Validaciones.TEXTO_MAX, "El nombre del alumno");
    }
```

Los cambios importantes son:

1. El **constructor no llama a los setters**: llama directamente a los métodos de validación. Un setter es público y una subclase podría sobrescribirlo; llamarlo desde el constructor sería riesgoso.
2. Los métodos de validación son **private static**: no se pueden sobrescribir y no dependen del estado del objeto.
3. Cada método **valida y devuelve el valor limpio** (por ejemplo, sin espacios al principio y al final).

Las reglas que se repiten en varias entidades (texto obligatorio con largo máximo, valor no nulo, entero en un rango) están en la clase de utilidades `Validaciones`:

util/Validaciones.java (resumido)

```
public final class Validaciones {

    public static final int TEXTO_MAX = 45; // Largo de los VARCHAR(45) de la base

    private Validaciones() { } // Clase de utilidades: no se instancia

    public static <T> T requerido(T valor, String mensaje) {
        if (valor == null) {
            throw new IllegalArgumentException(mensaje);
        }
        return valor;
    }

    public static String texto(String valor, int maximo, String campo) {
        if (valor == null || valor.trim().isEmpty()) {
            throw new IllegalArgumentException(campo + " no puede estar vacío.");
        }
        String limpio = valor.trim();
        if (limpio.length() > maximo) {
            throw new IllegalArgumentException(campo + " no puede superar los " + maximo + "
caracteres.");
        }
        return limpio;
    }
}
```

`final` impide heredar de la clase y el constructor `private` impide crear objetos: solo se usan sus métodos estáticos. `requerido` es un **método genérico** (`<T>`): sirve para cualquier tipo y devuelve el mismo tipo que recibe.

Capa de Repositorio (Repository): acceso a datos

El repositorio es una **interfaz**: no la implementamos nosotros. **Spring Data JPA** crea la implementación al arrancar la aplicación.

repository/AlumnoRepository.java

```
// save, findById, findAll, deleteById, etc. ya vienen heredados: NO hace falta redeclararlos.
@Repository
public interface AlumnoRepository extends JpaRepository<Alumno, Long> {

    // Query derivada: SELECT ... FROM alumno WHERE dni = ?
    Optional<Alumno> findByDni(String dni);

    // "curso.idCurso": navega la relación y filtra por el id del curso
    List<Alumno> findByCurso_IdCurso(Long idCurso, Sort orden);
}
```

`JpaRepository<Alumno, Long>` recibe dos tipos: la entidad que maneja y el tipo de su clave primaria. `@Repository` es de Spring y marca la capa de acceso a datos; en una interfaz que extiende `JpaRepository` es opcional, pero deja clara la intención.

Métodos que usamos

Método	Origen	Qué hace
`findById(id)`	Heredado	Busca por clave primaria. Devuelve un **Optional**: puede no haber resultado.
`findAll(pageable)`	Heredado	Devuelve una **página** de alumnos (listado paginado de a 10).
`save(alumno)`	Heredado	Inserta si el alumno es nuevo; actualiza si ya existe.
`saveAndFlush(alumno)`	Heredado	Igual que save, pero ejecuta el SQL **en ese momento**. Así un error de la base se puede capturar ahí mismo.
`delete(alumno)`	Heredado	Borra el alumno (y su domicilio, por el cascade).
`findByDni(dni)`	Derivado	Spring arma la consulta a partir del nombre: **find By Dni**.
`findByCurso_IdCurso(id, orden)`	Derivado	El guion bajo navega la relación: **alumno.curso.idCurso**.

El formulario: AlumnoForm (DTO)

Un **DTO** (*Data Transfer Object*) es una clase que solo transporta datos. Usamos `AlumnoForm` para recibir el formulario en lugar de la entidad, por tres motivos:

1. El formulario tiene datos que **no son del alumno**: la ciudad, la calle y la altura son del domicilio; el email y la contraseña, del usuario.
2. Recibe el **id del curso**, no un objeto Curso.
3. Evita que alguien agregue campos al formulario y modifique datos que no debería (por ejemplo, el id).

dto/AlumnoForm.java (resumido)

```java
@Getter
@Setter
@NoArgsConstructor
public class AlumnoForm {
    @NotBlank(message = "El DNI es obligatorio")
    @Pattern(regexp = "\\d{7,8}", message = "El DNI debe tener 7 u 8 dígitos")
    private String dni;

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 45, message = "El nombre no puede superar los 45 caracteres")
    private String nombre;

    @NotNull(message = "La fecha de nacimiento es obligatoria")
    @Past(message = "La fecha de nacimiento debe ser anterior a hoy")
    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    private LocalDate fechaNacimiento;

    @NotNull(message = "La altura es obligatoria")
    @Positive(message = "La altura debe ser mayor que cero")
    private Integer altura;

    @NotBlank(message = "El email es obligatorio")
    @Email(message = "Ingresá un email válido")
    private String email;

    // Solo se exige al crear. En edición, dejarlo vacío conserva la contraseña actual.
    private String password;

    @NotNull(message = "Seleccioná un curso")
    private Long idCurso;

    // Carga el formulario de edición con los datos actuales del alumno
    public static AlumnoForm desde(Alumno alumno) {
        AlumnoForm form = new AlumnoForm();
        form.setDni(alumno.getDni());
        form.setCiudad(alumno.getDomicilio().getCiudad());
        form.setEmail(alumno.getUsuario().getEmail());
        form.setIdCurso(alumno.getCurso().getIdCurso());
        // ... el resto de los campos, igual
        return form;
    }
}
```

Anotaciones de Bean Validation

Estas anotaciones son de **Jakarta Bean Validation** (paquete `jakarta.validation.constraints`). No hacen nada por sí solas: se ejecutan cuando el controlador recibe el formulario con `@Valid`.

Anotación	Se cumple cuando…
`@NotBlank`	El texto no es null, ni vacío, ni solo espacios.
`@NotNull`	El valor no es null (para fechas, números y el id del curso).

Anotación	Se cumple cuando…
`@Size(max = 45)`	El texto no supera los 45 caracteres, igual que la columna VARCHAR(45).
`@Pattern(regexp)`	El texto coincide con la expresión regular. **\d{7,8}** = 7 u 8 dígitos.
`@Past`	La fecha es anterior a hoy.
`@Positive`	El número es mayor que cero.
`@Email`	El texto tiene formato de email.
`@DateTimeFormat(iso = DATE)`	No valida: indica el formato **yyyy-MM-dd**, el único que entiende **<input type="date">**.

`@Getter`, `@Setter` y `@NoArgsConstructor` son de Lombok. Acá sí usamos `@Setter`: Spring necesita los setters para cargar los datos del formulario en el objeto.

Capa de Servicio (Service): las reglas de negocio

El servicio es el **corazón de la aplicación**: acá están las reglas que no se pueden expresar con una anotación, como "no puede haber dos alumnos con el mismo DNI" o "no se puede cambiar de curso a un alumno que ya tiene notas".

service/AlumnoService.java (encabezado y búsqueda)

```
@Service
@RequiredArgsConstructor
public class AlumnoService {

    private static final String MSJ_DUPLICADO = "Ya existe un alumno con ese DNI o una cuenta con
ese email.";

    private final AlumnoRepository alumnoRepository;
    private final CursoRepository cursoRepository;
    private final UsuarioRepository usuarioRepository;
    private final AlumnoMateriaRepository alumnoMateriaRepository;
    private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

    @Transactional(readOnly = true)
    public Alumno buscarPorId(Long id) {
        return alumnoRepository.findById(id)
                .orElseThrow(() -> new RecursoNoEncontradoException("No existe el alumno con id "
+ id));
    }
```

Anotación / clase	De dónde viene	Qué hace
`@Service`	Spring	Registra la clase en el **contenedor de Spring** (IoC). Spring crea una única instancia y la inyecta donde se pida.
`@RequiredArgsConstructor`	Lombok	Genera un constructor con todos los atributos **final**. Spring usa ese constructor para **inyectar los repositorios** (inyección por constructor, preferida a @Autowired).
`@Transactional`	Spring	Todo el método es una **transacción**: si termina bien se confirma (commit); si lanza una RuntimeException se deshace todo (rollback).
`@Transactional(readOnly = true)`	Spring	Transacción que solo lee: Hibernate no busca cambios para guardar y es más liviana.
`BCryptPasswordEncoder`	Spring Security Crypto	Hashea la contraseña. El hash no se puede revertir y nunca se guarda la contraseña en texto plano.

`orElseThrow` es un método de `Optional`: si el alumno existe lo devuelve, y si no, lanza la excepción que le pasamos con una lambda.

Crear un alumno

`service/AlumnoService.java: crear`

```
    @Transactional
    public Alumno crear(AlumnoForm form) {
        String dni = normalizar(form.getDni());
        String email = normalizar(form.getEmail()).toLowerCase(Locale.ROOT);
        // La regla de la contraseña se valida en texto plano, ANTES de hashearla
        Usuario.validarPasswordPlana(form.getPassword());
        validarUnicos(dni, email, null);
        Curso curso = buscarCurso(form.getIdCurso());
        try {
            Usuario usuario = usuarioRepository.save(new Usuario(email,
                    passwordEncoder.encode(form.getPassword()), Rol.ALUMNO));
            Domicilio domicilio = new Domicilio(form.getCiudad(), form.getCalle(),
form.getAltura());
            Alumno alumno = new Alumno(dni, form.getNombre(), form.getApellido(),
form.getFechaNacimiento(),
                    form.getFechaInscripcion(), domicilio, usuario, curso);
            return alumnoRepository.saveAndFlush(alumno);
        } catch (DataIntegrityViolationException e) {
            // Dos altas simultáneas con el mismo DNI/email pasan validarUnicos; la base (UNIQUE)
frena la segunda
            throw new ReglaDeNegocioException(MSJ_DUPLICADO);
        }
    }
```

Paso a paso:

1. **Normaliza** los datos: quita espacios y pasa el email a minúsculas.
2. Valida la **contraseña** (mínimo 8 caracteres) antes de hashearla.
3. Verifica que el **DNI y el email no estén repetidos** (*validarUnicos*).
4. Busca el **curso** elegido.
5. Crea primero el **usuario** (con la contraseña hasheada y rol ALUMNO), después el **domicilio** y por último el **alumno**. El domicilio se guarda solo, gracias al *cascade*.
6. Si la base rechaza algo (dos personas guardando el mismo DNI al mismo tiempo), convierte el error técnico en un mensaje claro.

Como el método es **@Transactional**, si algo falla en el paso 5 o 6 no queda un usuario "huérfano" en la base: se deshace todo.

Reglas de negocio

service/AlumnoService.java: reglas

```
    // "actual" es el alumno que se edita (null al crear): su propio DNI y email no cuentan como
repetidos
    private void validarUnicos(String dni, String email, Alumno actual) {
        alumnoRepository.findByDni(dni)
                .filter(a -> actual == null || !a.getIdAlumno().equals(actual.getIdAlumno()))
                .ifPresent(a -> { throw new ReglaDeNegocioException("Ya existe un alumno con el
DNI " + dni + "."); });
        usuarioRepository.findByEmail(email)
                .filter(u -> actual == null || !u.getId().equals(actual.getUsuario().getId()))
                .ifPresent(u -> { throw new ReglaDeNegocioException("Ya existe una cuenta con ese
email."); });
    }

    // Si ya tiene notas, cambiarlo de curso dejaría notas de materias que el curso nuevo quizás
no tiene.
    private void validarCambioDeCurso(Alumno alumno, Curso cursoNuevo) {
        boolean cambiaDeCurso = !alumno.getCurso().getIdCurso().equals(cursoNuevo.getIdCurso());
        if (cambiaDeCurso &&
alumnoMateriaRepository.existsByAlumno_IdAlumno(alumno.getIdAlumno())) {
            throw new ReglaDeNegocioException(
                    "No se puede cambiar de curso a un alumno que ya tiene notas cargadas.");
        }
    }
```

`filter` descarta al propio alumno cuando se está editando (si no, guardar sin cambiar el DNI daría "repetido"). `ifPresent` ejecuta la lambda solo si encontró otro alumno con ese DNI.

Eliminar un alumno

service/AlumnoService.java: eliminar

```
    @Transactional
    public void eliminar(Long id) {
        Alumno alumno = buscarPorId(id);
        try {
            alumnoRepository.delete(alumno);
            alumnoRepository.flush();
            // El domicilio se borra por el cascade; la cuenta se elimina explícitamente.
            usuarioRepository.delete(alumno.getUsuario());
            usuarioRepository.flush();
        } catch (DataIntegrityViolationException e) {
            throw new ReglaDeNegocioException("No se puede eliminar el alumno porque tiene notas
asociadas.");
        }
    }
```

Si el alumno tiene notas, la clave foránea de `alumno_materia` impide borrarlo. `flush()` fuerza a ejecutar el `DELETE` dentro del `try`, así el error se captura acá y se traduce a un mensaje entendible.

Capa de Controlador (Controller): conectar todo

El controlador recibe las peticiones del navegador, le pide el trabajo al servicio y decide qué vista mostrar. No tiene reglas de negocio: solo coordina.

controller/AlumnoController.java (resumido)

```
@Controller
@RequestMapping("/alumnos")
@RequiredArgsConstructor
public class AlumnoController {
    private static final String VISTA_FORM = "alumnos/form";
    private static final String REDIRECT_LISTA = "redirect:/alumnos";

    private final AlumnoService alumnoService;
    private final CursoService cursoService;

    @GetMapping
    public String listar(Model model,
            @PageableDefault(size = 10, sort = {"apellido", "nombre"}) Pageable pageable) {
        model.addAttribute("paginaAlumnos", alumnoService.obtenerAlumnos(pageable));
        return "alumnos/list";
    }

    @PostMapping
    public String crear(@Valid @ModelAttribute("alumnoForm") AlumnoForm form,
            BindingResult resultado, Model model, RedirectAttributes redirect) {
        if (!resultado.hasErrors()) {
            try {
                alumnoService.crear(form);
                redirect.addFlashAttribute("mensajeExito", "Alumno creado correctamente.");
                return REDIRECT_LISTA;
            } catch (ReglaDeNegocioException | IllegalArgumentException e) {
                resultado.reject("alumno.regla", e.getMessage());
            }
        }
        model.addAttribute("cursos", cursoService.obtenerTodos());
        return VISTA_FORM;
    }

    @PostMapping("/{id}/eliminar")
    public String eliminar(@PathVariable Long id, RedirectAttributes redirect) {
        try {
            alumnoService.eliminar(id);
            redirect.addFlashAttribute("mensajeExito", "Alumno eliminado correctamente.");
        } catch (ReglaDeNegocioException e) {
            redirect.addFlashAttribute("mensajeError", e.getMessage());
        }
        return REDIRECT_LISTA;
    }
}
```

Anotación / clase	Qué hace
`@Controller`	Marca la clase como controlador MVC: sus métodos devuelven el **nombre de una vista** Thymeleaf.
`@RequestMapping("/alumnos")`	Prefijo de todas las rutas de la clase.
`@GetMapping / @PostMapping`	Asocian un método a una URL y a un método HTTP. GET para mostrar; POST para modificar datos.
`@PathVariable`	Toma un valor de la URL: en **/alumnos/5/eliminar**, el 5.
`@ModelAttribute`	Arma un AlumnoForm con los campos del formulario (por nombre) y lo deja disponible en la vista.
`@Valid`	Ejecuta las anotaciones de Bean Validation del AlumnoForm.
`BindingResult`	Guarda los errores de validación. **Debe ir justo después** del parámetro con @Valid; si no, Spring lanza una excepción.
`Model`	Lleva datos del controlador a la vista con **addAttribute**.
`RedirectAttributes`	**addFlashAttribute** guarda un mensaje que sobrevive a una redirección y se muestra una sola vez.
`@PageableDefault`	Valores por defecto de la paginación: 10 por página, ordenados por apellido y nombre.

Después de un `POST` exitoso se devuelve `"redirect:/alumnos"`: el navegador hace un nuevo `GET` al listado. Este patrón se llama **Post/Redirect/Get** y evita que, al refrescar la página, se vuelva a enviar el formulario.

Rutas de Alumno

Método y URL	Método Java	Qué hace
`GET /alumnos`	listar	Listado paginado.
`GET /alumnos/{id}`	detalle	Ficha del alumno con las materias de su curso y sus notas.
`GET /alumnos/nuevo`	nuevo	Formulario vacío.
`POST /alumnos`	crear	Valida y crea. Si hay errores, vuelve al formulario.
`GET /alumnos/{id}/editar`	editar	Formulario con los datos actuales (AlumnoForm.desde).

Método y URL	Método Java	Qué hace
`POST /alumnos/{id}`	actualizar	Valida y actualiza.
`POST /alumnos/{id}/eliminar`	eliminar	Borra, o muestra por qué no se puede.

Capa de Vista (View): Thymeleaf

La vista `alumnos/form.html` sirve para crear y para editar: si el modelo trae `idAlumno`, es una edición.

templates/alumnos/form.html (resumido)

```
<form class="formulario" th:object="${alumnoForm}" method="post"
      th:action="${idAlumno != null} ? @{/alumnos/{id}(id=${idAlumno})} : @{/alumnos}">

    <!-- Errores del formulario completo (reglas de negocio) -->
    <p class="error-campo" th:each="err : ${#fields.globalErrors()}" th:text="${err}"></p>

    <div class="campo">
        <label for="dni">DNI</label>
        <input id="dni" type="text" inputmode="numeric" th:field="*{dni}" maxlength="8" required>
        <p class="error-campo" th:errors="*{dni}"></p>
    </div>

    <div class="campo">
        <label for="idCurso">Curso</label>
        <select id="idCurso" th:field="*{idCurso}" required>
            <option value="">Seleccioná un curso</option>
            <option th:each="curso : ${cursos}" th:value="${curso.idCurso}"
                    th:text="${curso.anio + '° ' + curso.division + ' - ' + curso.cicloLectivo}">
</option>
        </select>
    </div>

    <button type="submit" class="btn">Guardar</button>
</form>
```

Atributo	Qué hace
`th:object`	Indica el objeto del formulario (el AlumnoForm que dejó el controlador).
`th:field="*{dni}"`	Conecta el input con el campo **dni**: le pone **name**, **id** y el valor actual. El * significa "campo del th:object".
`th:errors="*{dni}"`	Muestra los mensajes de error de ese campo (los de Bean Validation).

Atributo	Qué hace
`#fields.globalErrors()`	Errores que no son de un campo: los que agregó el controlador con **resultado.reject(...)**.
`th:action`	URL a donde se envía el formulario. **@{...}** arma URLs con parámetros.
`th:each`	Repite el elemento por cada curso de la lista.
`th:text`	Reemplaza el contenido del elemento por el valor de la expresión.

Validaciones: cuatro barreras

Un mismo dato, como el DNI, se valida en varios lugares. No es repetición: cada barrera protege algo distinto, y si una falla o alguien la saltea, la siguiente lo frena.

1. **Navegador (HTML).** `required`, `maxlength="8"`, `type="date"`. Avisan al instante, pero se pueden saltear (por ejemplo, enviando la petición a mano).
2. **Formulario (Bean Validation).** `@NotBlank`, `@Pattern`, `@Past` en `AlumnoForm`, ejecutadas por `@Valid`. Los errores aparecen debajo de cada campo.
3. **Entidad y servicio (Java).** La entidad valida en su constructor y setters (`IllegalArgumentException`). El servicio valida lo que necesita consultar la base: DNI o email repetidos, cambio de curso con notas (`ReglaDeNegocioException`).
4. **Base de datos.** `NOT NULL`, `UNIQUE`, claves foráneas y `CHECK`. Es la última barrera: protege los datos aunque haya dos peticiones al mismo tiempo.

Ejemplo: un DNI repetido

Barrera	¿Lo detecta?	Por qué
Navegador	No	El navegador no conoce los datos guardados.
Bean Validation	No	El formato es válido: son 8 dígitos.
Servicio	Sí	**validarUnicos** busca el DNI en la base y lanza ReglaDeNegocioException.
Base de datos	Sí	Si dos personas lo guardan a la vez, el **UNIQUE** frena la segunda.

Excepciones

Excepciones propias

Creamos dos excepciones para distinguir los problemas de la aplicación. Las dos heredan de `RuntimeException` (excepciones **no chequeadas**): no obligan a poner `throws` en cada método y, al lanzarse dentro de un `@Transactional`, provocan el rollback automáticamente.

`exception/RecursoNoEncontradoException.java y ReglaDeNegocioException.java`

```
// Se lanza cuando se busca algo por id y no existe.
public class RecursoNoEncontradoException extends RuntimeException {
    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}

// Se lanza cuando una operación viola una regla (DNI repetido, borrar algo que tiene datos
asociados, etc.)
public class ReglaDeNegocioException extends RuntimeException {
    public ReglaDeNegocioException(String mensaje) {
        super(mensaje);
    }
}
```

Quién lanza cada excepción y quién la atrapa

Excepción	La lanza	La atrapa	Qué ve el usuario
`IllegalArgumentException`	La entidad (dato inválido)	El controlador, en crear y actualizar	El mensaje arriba del formulario.
`ReglaDeNegocioException`	El servicio (regla violada)	El controlador	El mensaje en el formulario, o en el listado al eliminar.
`RecursoNoEncontradoException`	El servicio (buscarPorId)	GlobalExceptionHandler	La página de error 404.

Excepción	La lanza	La atrapa	Qué ve el usuario
`DataIntegrityViolationException`	Spring, cuando la base rechaza algo	El servicio, que la traduce a ReglaDeNegocioException	Un mensaje claro, nunca el error de MySQL.

En el controlador usamos un **multi-catch**: `catch (ReglaDeNegocioException | IllegalArgumentException e)` atrapa las dos con un solo bloque, porque se tratan igual. `resultado.reject(...)` agrega el mensaje como error global del formulario, y la vista lo muestra con `#fields.globalErrors()`.

Manejo centralizado: GlobalExceptionHandler

Si una excepción "se escapa" de un controlador, la atrapa esta clase y muestra `error.html` con el diseño del sitio, en lugar del error técnico de Spring.

`exception/GlobalExceptionHandler.java (resumido)`

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ModelAndView manejarNoEncontrado(RecursoNoEncontradoException e) {
        return crearVista(HttpStatus.NOT_FOUND, "Recurso no encontrado", e.getMessage());
    }

    @ExceptionHandler(ReglaDeNegocioException.class)
    public ModelAndView manejarReglaDeNegocio(ReglaDeNegocioException e) {
        return crearVista(HttpStatus.CONFLICT, "No se pudo completar la operación",
e.getMessage());
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ModelAndView manejarIntegridad(DataIntegrityViolationException e) {
        // No se muestra el mensaje técnico de MySQL: expone nombres de tablas y restricciones
        return crearVista(HttpStatus.CONFLICT, "No se pudo completar la operación",
                "Los datos entran en conflicto con información ya guardada.");
    }

    private ModelAndView crearVista(HttpStatus status, String titulo, String mensaje) {
        ModelAndView mav = new ModelAndView("error");
        mav.setStatus(status); // El navegador recibe el código HTTP correcto (404, 409...)
        mav.addObject("status", status.value());
        mav.addObject("titulo", titulo);
        mav.addObject("mensaje", mensaje);
        return mav;
    }
}
```

Anotación / clase	Qué hace
`@ControllerAdvice`	La clase aplica a **todos** los controladores.
`@ExceptionHandler(X.class)`	El método se ejecuta cuando algún controlador deja escapar una excepción de tipo X.
`ModelAndView`	Junta en un solo objeto la vista (**error**) y los datos que necesita.
`HttpStatus`	Códigos HTTP: **404** no encontrado, **409** conflicto, **400** datos inválidos.

No hay un `@ExceptionHandler(Exception.class)` que atrape todo, a propósito: Spring Boot ya muestra `error.html` para cualquier otro error, y un "atrapa todo" convertiría los errores 404 y 400 en errores 500.

Errores de conversión

Si en un campo numérico llega texto (por ejemplo, "abc" en la altura), Spring no puede convertirlo y genera un error `typeMismatch`. Su mensaje lo definimos en `messages.properties`, que Spring Boot carga solo:

`src/main/resources/messages.properties`

```
typeMismatch.java.lang.Integer=Ingresá un número entero válido
typeMismatch.java.lang.Long=El valor elegido no es válido
typeMismatch.java.time.LocalDate=Ingresá una fecha válida
```

Resumen: crear un alumno de punta a punta

1. El usuario completa **/alumnos/nuevo** y presiona Guardar. El navegador controla `required` y `maxlength`.
2. Llega un **POST /alumnos**. Spring arma un `AlumnoForm` con los campos (`@ModelAttribute`) y ejecuta las validaciones (`@Valid`).
3. Si hay errores de formato, el controlador vuelve a mostrar el formulario con cada error debajo de su campo.
4. Si no, llama a `alumnoService.crear(form)`. El servicio abre una transacción, valida la contraseña, el DNI y el email, y busca el curso.
5. Crea el `Usuario`, el `Domicilio` y el `Alumno`; cada entidad valida sus datos en el constructor.
6. `AlumnoRepository.saveAndFlush` ejecuta el `INSERT`. La base verifica `NOT NULL`, `UNIQUE` y las claves foráneas.
7. Si una regla falla, la excepción vuelve al controlador, que muestra el mensaje en el formulario. La transacción se deshace.

8. Si todo sale bien, el controlador guarda el mensaje flash y redirige a **/alumnos**, donde aparece "Alumno creado correctamente."